



11.2 Student Guide: Asymmetric Encryption and Hashing

Overview

Today's class will expand on fundamental cryptography concepts, covering asymmetric encryption and hashing.

Class Objectives

By the end of class, you will be able to:

- Calculate the required number of symmetric and asymmetric keys based on the number of people exchanging secure messages.
- Use GPG to generate keys, and encrypt and decrypt private messages.
- Use hashes to validate the integrity of data.
- Use digital signatures to validate the authenticity of data.

Lab Environment

Lab details: You will continue to use your cryptography lab.

Slideshow

The slides for today can be viewed on Google Drive located here: [11.2 Slides](#)

01. Welcome and Review

In this lesson, we will continue learning and applying cryptography concepts.

Before we introduce new concepts, let's review those taught in the last class:

- Cryptography is the art and science of keeping information secure.
- Ciphers are cryptographic methods that disguise data by applying mathematical concepts called algorithms.
- Ciphers use encryption to convert plaintext into ciphertext, and use decryption to convert ciphertext back to plaintext.
- Ciphers use a key to specify how plaintext is converted to ciphertext and vice versa.
- The main cipher categories are block and stream ciphers.
- Stream ciphers apply their algorithms one character at a time, and block ciphers apply their algorithms to blocks of characters.
- Encoding is used to transform data to be used by another system, but is not designed to keep a message secret.
- Encoding, which doesn't use a key, is often used to transform digital text data into binary data, where encryption commonly takes place.
- The goals of cryptography are illustrated with the PAIN model, which stands for privacy, authentication, integrity, and non-repudiation.
- The security tradeoff is a cryptography concept that refers to the challenge of finding an encryption/decryption method that is fast and secure.
- Modern symmetric key algorithms attempt to solve this challenge by using a single key for encryption and decryption.
- DES, 3DES, and AES are modern symmetric key algorithms. AES is the most current and secure one in use today.
- OpenSSL is a command-line tool that can be used to apply symmetric key encryption.

02. Activity: Cryptography Refresher

 [M11.2 Activity File: Cryptography Refresher](#)

03. Activity Review: Cryptography Refresher

04. Introduction to Key Management and Exchange

In the previous activity, you used symmetric key encryption with OpenSSL.

- Symmetric key encryption has many benefits, such as:
 - Speed of encryption and decryption
 - Efficiency of encryption and decryption, with minimal computer resources required
 - Simple implementation for communication between two parties, as only one key is required

While symmetric key encryption has these benefits, it also comes with several disadvantages.

Disadvantage One: Secure Key Exchange

As we saw in the previous exercise, the first major disadvantage is insecure methods of distributing the keys.

- For example, unencrypted emails containing keys can be intercepted.

There are several other methods that can be used to exchange symmetric keys:

- Offline exchange, also known as an out-of-band exchange, can include mailing the key or calling the other party and reading them the key.
 - The vulnerabilities with this method could be intercepted mail or a tapped phone line.
- Diffie–Hellman key exchange is a method that uses mathematics to create a shared secret between two parties over a public channel, where the secret can't be seen even if the communication is captured.
 - The shared secret is the key and not a communication.

- While Diffie–Hellman is a complex method, you should just understand the basics — this is one method of exchange available to securely exchange keys over public channels.
- Additionally, Diffie–Hellman is often covered on security exams, such as Security+.
- The primary challenge of using Diffie–Hellman is the complexity of its implementation.
- If interested, you can research further using the following article and video:
 - [Diffie–Hellman Key Exchange \(Wikipedia\)](#)
 - [Secret Key Exchange \(Diffie-Hellman\) Video \(YouTube\)](#)

Disadvantage Two: Key Management

Another disadvantage of symmetric key cryptography is key management.

- Symmetric key cryptography between two parties only requires managing one key. But there needs to be a key for each combination of individuals, so the more combinations, the more keys required.
 - For example: If a small organization using symmetric key cryptography had four employees — Julie, Alice, Tim, and Bob — the following employee combinations would require six symmetric keys.
 - (Key 1) Julie, Alice
 - (Key 2) Julie, Tim
 - (Key 3) Julie, Bob
 - (Key 4) Alice, Tim
 - (Key 5) Alice, Bob
 - (Key 6) Tim, Bob
- This illustrates that four individuals would require six symmetric keys.
- Note that organizations typically have more than four employees. Therefore, each additional person adds a higher volume of keys to be managed.

While it would be challenging to write out all the different combinations for a larger organization, we can use a formula to calculate the number of required symmetric keys:

- Where N is the count of individuals

- $(N * (N-1)) / 2 = \text{count of symmetric keys}$
- For example, to figure out symmetric keys for an organization of seven people:
 - $(7 * 6) / 2 = 42/2 = 21$
- An organization of 1,000 employees would require managing almost half a million symmetric keys:
 - $(1000 * 999) / 2 = 499,500$
- This formula illustrates the challenge of key management that large organizations face when using symmetric encryption.
 - Additionally, note that calculating the number of keys required is often featured in security exams, such as Security+.

In the next section, we will introduce a solution that was created to address these disadvantages with symmetric key encryption, called asymmetric key encryption.

05. Asymmetric/Public Key Cryptography

Now that we covered the various disadvantages that come with symmetric key encryption, we can focus on methods to address them with asymmetric key encryption.

- Unlike symmetric encryption that uses one key, in asymmetric key encryption, each individual possesses a two-key pair.
- The two-key pair consists of a private key and a public key, which are linked together.
 - Public keys, as the name suggests, can be public and available for anyone to see.
 - Private keys need to be kept secret, as exposure could affect confidentiality of messages.
- Private and public keys are similar to symmetric keys in that they are typically a string of random numbers and letters.

Let's look at the following scenario to see how asymmetric key encryption works to encrypt a message:

Tim wants to send Julie his bank account number using asymmetric key encryption.

Step 1: Creating Key Pairs

- Since Julie needs to receive an encrypted message from Tim, she will have public and private key pairs created:
 - Julie's key pair: [Julie's private key] [Julie's public key]
- Julie's public keys are truly public: Julie puts her public key on her own website so anyone in the world can see it.

Step 2: Message Creation and Encryption

- Tim creates a plaintext message that contains his bank account number.
- He creates his message, goes to Julie's website, and gets her public key.
- He uses Julie's public key to encrypt his message.
[Tim's plain text secret message] encrypted with [Julie's public key] = [Tim's encrypted message]

Step 3: Message Exchange

- Tim sends his encrypted message to Julie.
 - He can send this message any way he chooses, even email or Slack, as it is now encrypted and can only be decrypted by Julie.
- Once it has been encrypted with Julie's public key, the only person who can decrypt it is Julie, as she owns the matching private key.

Step 4: Decryption

- Julie receives Tim's encrypted message and decrypts it with her matching private key.
- Julie can now see Tim's bank account number.
[Tim's encrypted message] decrypted with [Julie's private key] = [Tim's plaintext secret message]

Summary:

- In this scenario, only Julie's public and private keys were used.
- If Julie wanted to send an encrypted response to Tim, she would apply the same process but in reverse.

- Tim would need to create his own public and private key pair and put his public key on his website.
- Julie would encrypt her message with Tim's public key, which she obtained from his website.
- Tim would receive Julie's encrypted message and then decrypt it with his private key.
- Key pairs always have to be used together.
 - In other words, if a public key is used to encrypt a message, its matching private key will be used to decrypt the message.
- This scenario illustrates how asymmetric key encryption is used for confidentiality by keeping Tim's message secure from unauthorized parties.

We will now look at how this process of asymmetric key encryption addresses the disadvantages of symmetric key encryption.

Secure Key Exchange

In the scenario above, there was no need for Julie to find a secure way to get Tim her public key.

- Since the key was public and can be seen and accessed by anyone, a secure key exchange method isn't required.
- This is one of the biggest advantages of asymmetric key encryption.

Key Management

If Tim and Julie each needed to send messages to each other, four keys would be required for a secure communication:

- (1) [Tim's private key]
- (2) [Tim's public key]
- (3) [Julie's private key]
- (4) [Julie's public key]

As noted earlier, while symmetric keys use only one key for a secure exchange between two individuals, when more individuals are added in larger organizations, more and more keys are required, making keys difficult to manage.

- For example, in an organization of 12 employees, symmetric encryption would require 66 symmetric keys.
 - $(12 * 11) / 2 = 66$

For asymmetric encryption, each employee would only require their own key pair:

- The calculation is:
 - $N * 2$
- 12 employees would require 24 keys to be managed.
 - $(12 * 2) = 24$
- Note that for an organization of 12 employees, using asymmetric instead of symmetric would require 42 fewer keys.
 - $66 - 24 = 42$

This illustrates how asymmetric encryption addresses the key management issue, as far fewer keys are required for managing larger organizations.

RSA

Similar to symmetric encryptions' use of modern algorithms such as DES, 3DES, and AES, asymmetric encryption uses an algorithm called RSA.

- Introduced in 1977 and named after the last names of its creators: Rivest, Shamir, and Adelman
- The asymmetric algorithm that is still a worldwide standard today
- Works by factoring large prime numbers
 - If you are interested in further researching RSA, refer to the following source:
 - [RSA \(cryptosystem\) \(Wikipedia\)](#)

Summary

- Unlike symmetric encryption, which uses one key, asymmetric encryption uses two keys: one public and one private.
- Public keys are accessible by anyone, and private keys are only accessible by their owner.
- Public and private keys have to work together—if a public key encrypts a message, only the matching private key can decrypt that message.
- Asymmetric encryption addresses the challenges of symmetric encryption by:

- Not requiring a secure key exchange method
 - Using a smaller number of keys for larger organizations
 - The standard asymmetric algorithm used today is RSA.
-

06. Activity: Optimizing with Asymmetric Public Keys

 M11.2 Activity File: Optimizing with Asymmetric Keys

07. Activity Review: Optimizing with Asymmetric Public Keys

08. Applying Public Key Cryptography with GPG

Similar to symmetric key encryption, asymmetric encryption also has command-line tools to simplify the process of key creation, encryption, and decryption.

The next command-line tool we will demonstrate is called GPG.

- GPG stands for *GNU Privacy Guard*.
- It is a free software program available on many Linux distributions, which can run symmetric and asymmetric encryption algorithms.
- It can support a variety of algorithms such as 3DES, AES, and RSA.

We will demonstrate how to create a key pair and do asymmetric encryption and decryption with GPG.

- We will be using the same scenario of Tim sending his bank account number to Julie.

- This demonstration depicts two different individuals completing public key encryption.
- While not required, it is recommended to have two VMs open to simulate two individuals: one VM representing Tim and one representing Julie.

GPG Demonstration Setup

Begin by opening up your command line in your VM.

- GPG is already preinstalled on your VM for use and is also already installed in Ubuntu.
- We will first be conducting the activities of Julie, as she needs to create her key pair and provide her public key to Tim.
- Step 1: Creating the Key Pair

The first step of using asymmetric encryption is for Julie to generate her key pair.

- To create her public and private key, Julie will enter the following command:
`gpg --gen-key`
- Following this, Julie will be prompted to enter the following information:
 - Real name: What the user will name their key. We will use *Julie*.
 - Email address: The email associated with the key. We will use `julie@email.com`.
 - Change (N)ame, (E)mail, or (O)kay/(Q)uit?: Enter 0 to confirm your information.
 - Passphrase: A password to protect your private key. Any time the private key is used, the passphrase will be checked.
 - Select any passphrase, just take note of what it is, as it will be used later.
- A similar message will appear to show that the key pair has been created:

Unset

```
gpg: key D81710193A5FC56A marked as ultimately trusted
```

```
gpg: directory
'/home/instructor/.gnupg/openpgp-revocs.d' created

gpg: revocation certificate stored as
'/home/instructor/.gnupg/openpgp-revocs.d/C4A3CFC51B1318
FFD4D2C291D81710193A5FC56A.rev'

public and secret key created and signed
```

- If Julie wanted to validate the keys that are created, she would enter the following command:
gpg --list-keys
- This would return all the keys that are in Julie's keyring, as it is possible to have more than one key. A keyring is simply the storage of multiple keys.
 - A sample key ring with one key could look like the following:

Unset

```
/home/instructor/.gnupg/pubring.kbx
```

```
pub    rsa3072 2019-12-11 [SC] [expires: 2021-12-10]
```

```
C4A3CFC51B1318FFD4D2C291D81710193A5FC56A
```

```
uid          [ultimate] julie <julie@email.com>
```

```
sub    rsa3072 2019-12-11 [E] [expires: 2021-12-10]
```

- **Step 2: Exporting and Importing Keys**

The sender of the message needs to have the receiver's public key to encrypt the message.

Julie needs to export her public key to make it public, so Tim can use it.

- Exporting puts the public key in a format that can be shared and used to encrypt.
- Julie would use the following command to export her public key:

```
gpg --armor --output julie.gpg --export julie@email.com
```

- `gpg`: This is the command to run GPG.
- `--armor`: This puts the key in an ASCII format.
- `--output julie.gpg`: This creates the public key in an accessible format.

In this case, we named the key `julie.gpg`.

- `--export julie@email.com`: This references which key to use from the keyring. It is referenced by the email.
- Run the command to export the public key in a format that can be shared.
- To view the key, run the following command:
 - `cat julie.gpg`
- The results should resemble the following format:
 - Note: This is a shortened example.

Unset

-----BEGIN PGP PUBLIC KEY BLOCK-----

oCN2AghQUdgu5yBVAmPAx7hatvcMBR1X6NqJN4wStLB210vHdgT2VbiH
UtwkGvbJ

Hsui9eTR7bBY1YgP8PcGFjeMZ5+C7E94uYeksbwMzFWGE79M3kqEi1tg
kDZTN/T8

8031qQUgDCCbUnuwpW5pYJ2BconeNBHAZNSKsg+9U3DfCazRpky89be6
W7WtjDGs

iFo5PEjBTvCJJXHvDgn2W7I7U0MW0220gyCT/Ja/eKad5GKTeMj0C4ER
Twvha00N

```
-----END PGP PUBLIC KEY BLOCK-----
```

- For our scenario, Julie would put this public key on her website, or she could directly share it with Tim.
- Next, we will illustrate the steps that Tim needs to take:
 - First, once Tim gets this key and saves it in his current directory, he will need to import it into his keyring.
 - The command for Tim to import this key is:
`gpg --import julie.gpg`
 - Run this command. To confirm that Julie's key has been imported, the `list-keys` command can be run again:
`gpg --list-keys`
 - Show that the results display that Julie's public key was added to Tim's keyring.
 - Tim has not yet created any keys. We will do that later.

Unset

```
pub  rsa3072 2019-12-03 [SC] [expires:
2021-12-02]

39B2BD6C93E1E63E8C004183FE91AF7A7B4EC267

uid  [ultimate] Julie <julie@email.com>
```

```
sub    rsa3072 2019-12-03 [E] [expires: 2021-12-02]
```

- **Step 3: Encryption**

- Tim now has Julie's public key in his keyring, so he is ready to create a message and encrypt it.

- Tim will want to create a file that will contain a message, so we'll use the following echo command to create a file called `Tims_plainmessage.txt`.

```
echo "Hi Julie, my bank account number is 2783492" >  
Tims_plainmessage.txt
```

- The next step is to use Julie's public key to encrypt `Tims_plainmessage.txt`.

- We will use the following command:

```
gpg --armor --output Tims_encryptedmessage.txt --encrypt  
--recipient julie@email.com Tims_plainmessage.txt
```

- `gpg`: This is the command to run GPG.
- `--armor`: This puts the encrypted message in an ASCII format.
- `--output Tims_encryptedmessage.txt`: This is the command for the output file, which creates the name of the encrypted file.
- `--encrypt`: This tells GPG to encrypt.
- `--recipient julie@email.com`: This tells GPG which public key to use based on the email address of the key.
- `Tims_plainmessage.txt`: This specifies for GPG which plaintext file to encrypt.
- Run the command to create a file that has ciphertext called `Tims_encryptedmessage.txt`.

- Run a preview command to illustrate the file has now been encrypted:

```
cat Tims_encryptedmessage.txt
```

- The encrypted message should look like the following:

Unset

-----BEGIN PGP MESSAGE-----

hQGMA1p4Le4c2oCaAQv+MT2ghzg9RYymSIxnbwe41LpOPx76mA9f6mQY
Z077c/Ij

u14kEgfaVM9PxxBw8KpEkg5NvmBVPAfxFbFrcLoKB8lVW8MTpp3mQ8r0
257PNORK

bQ0C+HHktQN3AJrsgN/Oj40duM+hMtnPUdWa0X7u00KRFW9r5CbuYga1
34EzoHG3

-----END PGP MESSAGE-----

- **Step 4: Decryption**
- The last step is for Tim to send his encrypted message over to Julie so that she can decrypt it with her private key.
 - Once Julie receives Tim's encrypted message, she will save it in a directory and then run decryption commands against the file.

- The command that Julie will use to decrypt Tim's encrypted message is:
`gpg --output Tims_decrypted_message --decrypt Tims_encryptedmessage.txt`
 - `gpg`: This is the command to run `gpg`.
 - `--output Tims_decrypted_message`: This creates an output file, which is the decrypted message.
 - `--decrypt Tims_encryptedmessage.txt`: This is indicating to decrypt and what file to decrypt.
- Run the command. It just decrypted Tim's message and placed the results into a file called `Tims_decrypted_message`.
- Preview the decrypted file by running:
 - `cat Tims_decrypted_message`
- This shows that now Julie can see Tim's plaintext message:
 - "Hi Julie, my bank account number is 2783492."

Summary:

- This walkthrough illustrated the steps for asymmetric encryption and decryption.
- While it may seem complicated at first, the best way to understand them is to apply them yourself.

09. Activity: GPG

 M11.2 Activity File: GPG

10. Activity Review: GPG

11. Break

12. Hashing and Data Integrity

Over the past several sections, we covered how cryptography can be applied to protect the privacy and confidentiality of data with symmetric and asymmetric encryption.

- Cryptography can also be applied to protect the integrity of data.
 - For example, a police investigator wants to present a computer file as evidence in court.
 - The police investigator will want to also prove the file hasn't been modified or tampered with since it was collected.
 - This is to prove to the court that the integrity of the data has been protected.

Hashing is a cryptographic method for proving the integrity of data.

Hashing, like encryption, uses mathematical algorithms, called hashing algorithms, to take data and generate a unique output.

We will show how hashing works with a simple hashing demonstration:

- We will use the following plaintext sentence:
 - I Love Cryptography!
- When we apply a hashing algorithm to this sentence, we get:
 - 676e4bff90a76853bda00773f7ad4bed
- This is an MD5 hashing algorithm.
- At first glance, hashing may look similar to encryption, as you are unable to understand the resulting message.

It is important to understand that hashing and encryption actually have several significant differences:

1. Encryption takes plaintext and converts it to ciphertext with a key and an algorithm.
 - Hashing takes plaintext and converts it into a message digest with an algorithm and no key.

- A message digest, also known as a fingerprint, hash, or checksum, is a unique identifier of the plaintext that is outputted from a hashing algorithm.
 - In the example, the message digest was
676e4bff90a76853bda00773f7ad4bed.
2. With encryption, plaintext gets converted into ciphertext, and then gets returned back to plaintext with decryption.
 - With hashing, once the plaintext gets converted into a message digest, it cannot be converted back into plaintext.
 - In other words, 676e4bff90a76853bda00773f7ad4bed $\neq$ "I Love Cryptography!"
 - This is why hashing is called a one-way function.
 3. With encryption, the input can be any length and the output can be any length with a specific algorithm.
 - With hashing, the input can be any length and the output has a fixed length with a specific hashing algorithm.
 4. The primary goal of encryption is privacy. The primary goal of hashing is integrity. Integrity is accomplished with hashing:
 - If a small change is made to the sentence, the same hashing algorithm would produce a completely different message digest.
 - This would indicate the integrity of the data was compromised.
 - For example, if we add a second exclamation point to our sentence—I Love Cryptography!!—and apply the same hashing algorithm, the message digest would be significantly different, indicating the data had been modified:
 - 4e6fc433ff57a6c4a854cbbcff65f61a
 - I Love Cryptography! = 676e4bff90a76853bda00773f7ad4bed
 - I Love Cryptography!! = 4e6fc433ff57a6c4a854cbbcff65f61a
- While the above example shows a simple sentence, cybersecurity professionals apply the same hashing concept to much larger sets of data such as files, website code, emails, databases, and computer hard drives.

- In these larger sets of data, even the smallest change would result in a significantly different message digest.

We covered that encryption has common algorithms, such as AES, DES, and RSA.

- Hashing has several hashing algorithms we should be familiar with, such as:
 - SHA (secure hash algorithms), which include its successors: SHA1 and SHA2.
 - SHA2 has variations, with different security strengths: SHA-256 and SHA-512.
 - MD (message digest) has several variations: MD2, MD4, and MD5.
 - LM and NTLM are hashes used by Windows.

While each of these hashing algorithms have different mathematical formulas, they are all used to convert plaintext into a message digest.

- The easiest way to get familiar with the behavior of the various hash algorithms is to use them!

Creating Hashes on the Command Line

While these hashing algorithms have complex mathematical formulas, there are command-line tools that easily create message digests with a simple terminal command.

We will be using two command line tools to create hashes: `md5sum` and `sha256sum`.

- Running `md5sum` uses the MD5 hashing algorithm to create a message digest from a plaintext message.
- Running `sha256sum` uses the SHA-256 hashing algorithm to create a message digest from a plaintext message.

We will apply these commands against a new file called `secretmessage.txt`.

- First, create a basic message inside the file, such as: "This is my first hashing activity."
- Run the following command:
 - `echo "This is my first hashing activity" > secretmessage.txt`

To create an MD5 message digest of the new file, we will use the following command:

- `md5sum secretmessage.txt > hashes.txt`
 - `md5sum`: The terminal command to run the MD5 algorithm

- `secretmessage.txt`: The file to be hashed
- `> hashes.txt`: The output file where the message digest is placed
- This last command is optional. If removed, the message digest will display back on the command line.
- Run the command. A file containing the message digest, called `hashes.txt`, has been created.
 - Preview the file by running:
 - `cat hashes.txt`
 - The results should show:
 - `bdbd28dbb5f51abb282ecd0b9daa3e69 secretmessage.txt`
 - This shows the message digest and the file from where the message digest originated.

`md5sum` can also be used to check the integrity of the file.

- In other words, it will check to see if the file has been modified since the message digest was created.

We will demonstrate how to do this by making a change to the `secretmessage.txt` file.

- Modify the `secretmessage.txt` file by overwriting it with the following command.
 - `echo "This is my second hashing activity" > secretmessage.txt`
- Next, we will run the `md5sum` check command to validate the integrity of the file.
 - `md5sum -c hashes.txt > md5check.txt`
 - `md5sum`: The terminal command to run the MD5 algorithm
 - `-c`: The option to have `md5sum` check the hashes
 - `hashes.txt`: The file the check is being run against
 - `> md5check.txt`: The output file where the results of the check are placed
 - The command works by:
 - Looking in the `hashes.txt` file for the file name and associated message digest
 - Running the MD5 hash again on the files in the current directory and checking to see if the message digests still match

- Run the command. It immediately confirms that one of the message digests (or checksums) didn't match:
 - `md5sum: WARNING: 1 computed checksum did NOT match`
- Preview the output file to confirm which file failed the check:
 - `cat md5check.txt`
- The results should clearly show the file that was modified:
 - `secretmessage.txt: FAILED`

`md5sum` uses the MD5 hashing algorithm and the exact same steps can be accomplished with the SHA-256 algorithm by simply replacing:

- `md5sum` with `sha256sum`

Summary:

- While encryption is used for confidentiality, hashing is used for integrity.
- Hashing uses hashing algorithms to create message digests from plaintext.
- A small change in the plaintext causes a significant change in the message digest.
- Common hashing algorithms are SHA1, SHA2, and MD5.
- `md5sum` and `sha256sum` are command-line tools used to create message digests and check the integrity of files.

13. Activity: Generating Hashes

 M11.2 Activity File: Generating Hashes

14. Activity Review: Generating Hashes

15. Digital Signatures

So far, we have covered how the cryptographic process of encryption is used for confidentiality and the process of hashing is used to verify integrity.

- Cryptography can also be used to validate authenticity.
 - For example, an accounting representative at an organization receives a message from their CEO to wire funds immediately to a location in Asia.
 - The accounting representative needs to validate the message is authentic and actually from the CEO.
 - If the accounting representative wires the funds before realizing the message is from a scammer, it could have a serious financial impact on the organization.
- There is a cryptographic process used to verify authenticity that can assist with this example and much more called digital signatures.
 - A digital signature is a mathematical scheme for verifying the authenticity of digital data.
 - While its primary purpose is for authenticity, it can also provide non-repudiation and integrity.
 - In the United States and several other countries, digital signatures are considered legally binding, similar to a standard signature.

Like asymmetric encryption, digital signatures also use public key cryptography, except digital signatures use public and private keys in reverse.

Digital Signature Walkthrough

We will illustrate how digital signatures work with public key cryptography using the following scenario:

Tim wants to send Julie a message that says "Transfer \$500 to the account I provided to you." He also wants to digitally sign the message so Julie knows it originated from him and is authentic.

- Step 1: Key Creation
 - We previously created a key pair for Julie, but we now will create a key pair for Tim.

- Step 2: Creating the Message
 - Tim places his message in a file called `Tims_message.txt`.
- Step 3: Signing the Message
 - Tim signs the message with his private key to create a digital signature.
`[Tims_message.txt]` signed with `[Tim's private key]` = `[Tim's digital signature]`
 - This step is critical to understand:
 - Encryption uses the recipient's public key to encrypt.
 - Digital signatures use the sender's private key to sign.
- Step 4: Sending the Message and Signature
 - Tim sends the digital signature to Julie along with his plaintext message.
 - Julie will receive `[Tims_message.txt]` and `[Tim's digital signature]`
- Step 5: Validating the Signature
 - After Julie receives Tim's digital signature and Tim's message, she will grab Tim's public key from his website to validate the signature.
 - `[Tim's digital signature]` + `[Tims_message.txt]` validated with `[Tim's public key]`
 - Julie will use a signature-validation program, such as GPG, to validate the message is authentic based on the signature.
 - The program will either:
 - Confirm the message is authentic and came from Tim.
 - Deny, if the message is inauthentic and didn't come from Tim.

This scenario shows how digital signatures can also be used for integrity and non-repudiation:

- Integrity: If Tim's message was modified, the digital signature validation would fail.
- Non-repudiation: Tim would not be able to deny he was the one who signed the message.

This walkthrough illustrates one of the ways digital signatures can be used to verify a message's authenticity. This method is called a detached signature.

- It is called a detached signature because the message and the signature are sent separately and not attached to each other.
 - Other digital signature methods can include:
 - All at once: This refers to a signature appended to an encrypted message.
 - Clearsigned: This refers to a signature appended to an unencrypted message.
 - Signed hash: This means that instead of signing a message, a hash is created first and the hash is signed for verification.
-

16. Signing with GPG

In the next demonstration, we will illustrate how to apply a detached digital signature with the same GPG program used for GPG asymmetric encryption.

- We will conduct this demonstration with the scenario of Tim signing a message for Julie stating: "Transfer \$500 to the account I provided to you."

Walkthrough

- Step 1: Key Creation
 - Since we've only created a key pair for Julie, we will now create one for Tim with the same commands:
 - `gpg --gen-key`
 - After pressing enter, Tim will be prompted with the same questions:
 - Real name: What the user will name their key. We will use *Tim Doe*.
 - Email address: The email associated with the key. We will use `tim@email.com`.
 - Change (N)ame, (E)mail, or (O)kay/(Q)uit?: Enter 0 to confirm your information.
 - Passphrase: A password to protect your private key. Any time the private key is used, the passphrase will be checked.

- Select any passphrase, just take note of what it is, as it will be used later.
 - Tim will also export his public key with the following command:
 - `gpg --armor --output tim.gpg --export tim@email.com`
 - Tim will put his public key, `tim.gpg`, on his website for anyone to use.
- Step 2: Creating the Message
 - Tim will place his message in a file called `Tims_message.txt`.
 - Run the following command to create the file containing the message:

```
echo "Transfer \"$500 to the account I provided to you" >
Tims_message.txt
```
 - The `\` is necessary to prevent the system from treating the `$` as a variable.
- Step 3: Signing the Message
 - Next, Tim will sign the message with his private key to create a detached digital signature.
 - Use the following command to sign the message:
 - `gpg --output Tims_signature --armor --detach-sig Tims_message.txt`
 - `gpg` runs the GPG command.
 - `--output Tims_signature` specifies the output file that contains the digital signature.
 - `--armor` outputs the signature in an ASCII format.
 - `--detach-sig Tims_message.txt` specifies that a detached signature will be created against the file `Tims_message.txt`.
- Run the command. Since digital signatures automatically use your private key, GPG will prompt you to put in the password used to create your key pair.
- Enter the password. A separate digital signature called `Tims_signature` has just been created.

- Preview the signature by running the following command:
`cat Tims_signature`
- The signature should look like the following:

Unset

-----BEGIN PGP SIGNATURE-----

iQGzBAABCgAdFiEE0bK9bJPh5j6MAEGD/pGventOwmcFA13pGKoACgkQ
/pGvent0

wmdPfQv8CigGztcvrdZrJVr91mPiLL5cry7nKYDAsRqkyIDltiJMxtg
gVbCtSPm

YLfqZATWYofBWdE4wkpmeYE96gXTeJP4VVNUUpwnshg1A1q0att10S+r1
v6N73g4V

-----END PGP SIGNATURE-----

- Step 4: Sending the Message and Signature
 - Tim will send the digital signature file to Julie along with his plaintext message.
 - Julie will receive `Tims_signature` and `Tims_message.txt`.
 - Julie needs to place both files in the directory where she is running the verification.
- Step 5: Validating the Signature
 - After Julie receives Tim's digital signature and Tim's message, she will grab Tim's public key from his website to validate the signature.
 - For Julie to verify Tim's message, Julie will need to import Tim's public key that she got from his website with the following command:
 - `gpg --import tim.gpg`
 - For Julie to validate Tim's signature, the following command will be run:
 - `gpg --verify Tims_signature Tims_message.txt`
 - Run the command to verify that the signature is valid:

Unset

```
gpg: Signature made Thu 05 Dec 2019 09:48:10 AM EST
```

```
gpg:                               using RSA key
39B2BD6C93E1E63E8C004183FE91AF7A7B4EC267
```

```
gpg: Good signature from "Tim <tim@email.com>"  
[ultimate]
```

We will now show how digital signatures with GPG can be used to not only validate the authenticity, but also the integrity.

- Step 1: Modifying the Message
 - We will run the following command to modify the amount of money specified in the message:

```
echo "Transfer \"$34,547 to the account I provided to you" >  
Tims_message.txt
```
- Step 2: Validating the Signature Again
 - Run `gpg --verify Tims_signature Tims_message.txt`
 - This now clearly shows the signature failed.

Unset

```
gpg: Signature made Thu 05 Dec 2019 09:48:10 AM EST  
  
gpg:                using RSA key  
39B2BD6C93E1E63E8C004183FE91AF7A7B4EC267  
  
gpg: BAD signature from "Tim <tim@email.com>" [ultimate]
```

This walkthrough showed how to do a detached digital signature. There are other methods of digital signatures that can also be accomplished with GPG.

- If you are interested in learning other digital signature methods, you can visit [The GNU Privacy Handbook entry on making and verifying signatures](#).

Summary:

- While encryption is used for confidentiality and hashing is used for integrity, digital signatures are used for authentication, non-repudiation, and integrity.
- A digital signature is a mathematical scheme for verifying the authenticity of digital data.
- Like encryption, digital signatures also use public key cryptography.
 - However, in the case of digital signatures, a user uses their own private key to sign a document, and the public key would be used by other users to validate the signature.
- There are several types of digital signatures available: detached signatures, all at once, and clearsigned.
- GPG is a command-line tool that can apply these digital signature methods.

17. Activity: Digital Signatures

 [M11.2 Activity File: Digital Signatures](#)

18. Activity Review: Digital Signatures
